

DIM0119 — Estrutura de Dados Básicas I

Lista de Exercícios — Listas Encadeadas

Disciplina	DIM0119 — Estrutura de Dados Básicas I
Professor	Marcus Vinicius Alencar Terra — DIMAp / UFRN
Tema	Listas Encadeadas — Simples, Circular e Duplamente Encadeada
Exercícios	5 exercícios — linguagem C++

Instruções Gerais

- Todos os exercícios devem ser resolvidos em **linguagem C++**, utilizando as classes definidas na seção de Código Base.
- Implemente apenas o corpo dos métodos — a assinatura e os tipos já estão fornecidos.
- Respeite as restrições de complexidade indicadas em cada exercício.

Código Base — Classes e Tipos

As três variantes de lista abaixo são a base para todos os exercícios. Inclua-as no topo do seu arquivo `.cpp`.

```
C/C++
#include <iostream>
#include <stdexcept>
using namespace std;

// ——— Lista Encadeada Simples ———
struct Node {
    int data;
    Node* next;
    Node(int d) : data(d), next(nullptr) {}
};

struct LinkedList {
    Node* head = nullptr;
    int size = 0;
};

// ——— Lista Duplamente Encadeada ———
struct DNode {
    int data;
    int priority; // 0 = normal, 1 = alta
```

```

DNode* prev;
DNode* next;
DNode(int d, int p) : data(d), priority(p),
                    prev(nullptr), next(nullptr) {}
};

struct DLinkedList {
    DNode* head    = nullptr;
    DNode* tail    = nullptr;
    int     size    = 0;
};

// ——— Lista Circular (reusa Node) ———
struct Circlist {
    Node* tail = nullptr; // tail->next == head
    int     size = 0;
};

```

01 `inverte()` — Inverter lista encadeada simples in-place | $O(n)$ / $O(1)$

Enunciado: Implemente a função `void inverte(LinkedList& l)` que inverte a **ordem dos nós** da lista encadeada simples **in-place**, sem criar novos nós e sem usar vetor auxiliar. Receba a lista por referência para poder alterar o ponteiro `head`.

Tempo $O(n)$ — percorre a lista uma única vez

Espaço $O(1)$ — apenas ponteiros locais (sem recursão)

Exemplo **Entrada:** `1 → 2 → 3 → 4 → nullptr` → **Saída:** `4 → 3 → 2 → 1 → nullptr`

Esqueleto a completar

```

void inverte(LinkedList& l) {
}

```

02 rotaciona() — Rotacionar lista circular k posições | $O(k \bmod n)$

Enunciado: Implemente `void rotaciona(CircList& c, int k)` que rotaciona a lista circular **k posições à direita**. A lista é representada pelo ponteiro `tail` (onde `tail->next == head`).

Após a rotação, o elemento que antes estava k posições antes do tail passa a ser o novo tail — **basta avançar o ponteiro tail k posições**, sem mover nenhum nó.

Tempo $O(k \bmod n)$ — versão otimizada

Espaço $O(1)$ — nenhuma alocação extra

Exemplo $k=2, n=5$: `[1→2→3→4→5]` → `[4→5→1→2→3]` (head passa a ser [4])

Esqueleto a completar

```
void rotaciona(CircList& c, int k) {  
}
```

03 enqueue_priority() — Fila de Prioridade | O(1) / O(1)

Enunciado: Implemente `void enqueue_priority(DLinkedList& l, int data, int priority)` em **O(1) de tempo** e **O(1) de espaço**. Altere a struct `DLinkedList` conforme necessário.

Nós com `priority=1` (alta) devem vir **antes** dos nós com `priority=0` (normal). FIFO é preservado dentro de cada grupo.

Tempo	<code>O(1)</code> — acesso direto via ponteiros
--------------	---

Espaço	<code>O(1)</code> — nenhuma estrutura auxiliar
---------------	--

Exemplo	<code>normal(A), normal(B), alta(C), normal(D), alta(E) → fila: C → E → A → B → D</code>
----------------	--

Esqueleto a completar

```
void enqueue_priority(DLinkedList& l, int data, int priority) {  
}
```

04 merge() — Mesclar duas listas ordenadas | $O(n+m)$ / $O(1)$

Enunciado: Implemente `LinkedList merge(LinkedList& a, LinkedList& b)` que mescla duas listas ordenadas em ordem **crescente, reutilizando os nós existentes** — ajuste apenas ponteiros `next`.

Tempo $O(n+m)$ — cada nó de ambas as listas é visitado uma única vez

Espaço $O(1)$ — nenhum nó extra no heap

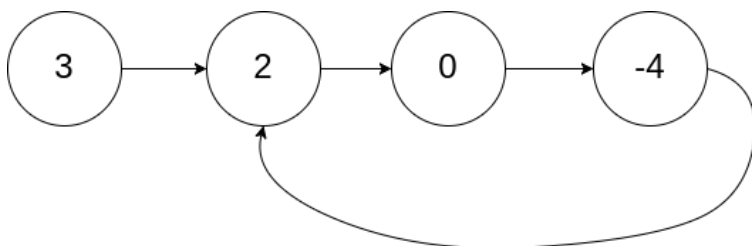
Exemplo $[1 \rightarrow 3 \rightarrow 5 \rightarrow 7] + [2 \rightarrow 4 \rightarrow 6] \rightarrow [1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7]$

Esqueleto a completar

```
LinkedList merge(LinkedList& a, LinkedList& b) {  
}
```

05 tem_ciclo() — Detectar ciclo (Algoritmo de Floyd) | $O(n)$ / $O(1)$

Enunciado: Implemente `bool tem_ciclo(const LinkedList& l)` que retorna `true` se a lista contiver um ciclo e `false` caso contrário, usando **$O(1)$ de espaço extra** — sem marcar nós, sem estrutura auxiliar.



Tempo $O(n)$ — no máximo n iterações para detectar o ciclo

Espaço $O(1)$ — apenas ponteiros auxiliares

Esqueleto a completar

```
bool tem_ciclo(const LinkedList& l) {  
}
```

Resumo de Complexidade

Exercício	Função	Tipo de Lista	Tempo	Espaço	Ideia-chave
01	inverte()	Simples	$O(n)$	$O(1)$	3 ponteiros: prev, curr, next
02	rotaciona()	Circular	$O(k \bmod n)$	$O(1)$	Avança tail k posições
03	enqueue_priority()	Dupla	$O(1)$	$O(1)$	ponteiro priority_tail
04	merge()	Simples	$O(n+m)$	$O(1)$	Nó sentinela (dummy)
05	tem_ciclo()	Simples	$O(n)$	$O(1)$	Floyd: lento $\times 1$, rápido $\times 2$